OSS License Analyzer - Comprehensive Architectural Breakdown

Complete Technical Documentation

Author: AIG Engineering Team

Date: December 2025

OSS License Analyzer - Comprehensive Architectural Breakdown

Executive Summary

The OSS License Analyzer is a Node.js-based web application that analyzes GitHub repositories for open-source license compliance. It uses the OSS Review Toolkit (ORT) via Docker to extract dependency information, evaluates them against a configurable policy (Policy-as-Code), and stores compliance results in a SQLite database. The frontend uses Alpine.js for reactive UI components.

---

1. Complete File Structure

```
oss-license-analyzer/
├── src/
│   ├── server.js                          # Application entry point
│   ├── config/
│   │   ├── database.js                    # Sequelize SQLite configuration
│   │   ├── logger.js                      # Winston logger setup
│   │   ├── policy.js                      # License policy configuration
│   │   └── docker.js                      # Docker/ORT configuration
│   ├── models/
│   │   ├── index.js                       # Model associations
│   │   ├── ComplianceRun.js              # Compliance run entity
│   │   └── PolicyComplianceResult.js     # Individual dependency verdict
│   ├── routes/
│   │   ├── github.js                      # GitHub repository analysis
│   │   ├── spdx.js                        # SPDX license queries
```

```
│   │   ├── policy.js                      # Policy compliance checks
│   │   ├── complianceRuns.js             # Compliance run management
│   │   └── shutdown.js                    # Graceful shutdown
│   ├── services/
│   │   ├── GitHubRepoService.js          # GitHub API interaction
│   │   ├── LicenseAnalyzerService.js     # ORT Docker execution
│   │   ├── ComplianceEvaluator.js        # Policy evaluation logic
│   │   ├── ComplianceRunService.js       # Database persistence
│   │   └── SpdxLicenseRegistry.js        # SPDX license data
│   ├── utils/
│   │   ├── GitHubApiClient.js            # HTTP client for GitHub API
│   │   └── SpdxLicenseSearchUtil.js      # License search utilities
│   ├── middleware/
│   │   ├── errorHandler.js               # Global error handling
│   │   └── shutdownAuth.js               # API key authentication
│   └── data/
│       └── spdx/
│           └── backup-license.json       # SPDX license database
├── public/
│   ├── index.html                         # Main analysis page
│   ├── licenses.html                      # SPDX license browser
│   ├── paac-results.html                 # Compliance results viewer
│   ├── compliance-run.html               # Compliance tracker
│   ├── shutdown.html                      # Shutdown control panel
│   ├── css/
│   │   └── aig-bootstrap-override.css    # Custom styling
│   └── images/
│       └── aig-logo.png                   # AIG branding
├── data/
│   └── paacdb.sqlite                      # SQLite database
├── logs/
│   ├── app.log                            # Application logs
│   └── error.log                          # Error logs
├── package.json                           # Node.js dependencies
├── .env                                   # Environment configuration
└── README.md                              # Documentation
```

---

2. Server Initialization & Entry Point

`src/server.js`

Initialization Sequence:

1. **Load Environment Variables** - `.env` file loaded via dotenv
2. **Create Express App** - Configure middleware stack
3. **Initialize Database** - Test connection and sync schema
4. **Load SPDX Registry** - Load 500+ SPDX licenses into memory
5. **Start HTTP Server** - Listen on port 8080 (default)

Middleware Stack (in order):

```
1. helmet()                    // Security headers
2. cors()                      // Cross-origin resource sharing
3. express.json()              // Parse JSON bodies
4. express.urlencoded()        // Parse URL-encoded bodies
5. Request Logger              // Log all incoming requests
6. express.static()            // Serve public/ directory
7. Route Handlers              // API endpoints
8. errorHandler                // Global error handling
```

Registered Routes:
- `/api/github/*` → GitHub repository analysis
- `/api/spdx/*` → SPDX license queries
- `/api/policy/*` → Policy compliance evaluation
- `/api/compliance-runs/*` → Compliance run management
- `/api/shutdown` → Graceful shutdown
- `/health` → Health check endpoint

---

3. Route Definitions & Mappings

3.1 GitHub Analysis Routes (`/api/github`)

Method	Endpoint	Controller	Purpose
POST		`githubRoutes`	

Method	Endpoint	Controller	Purpose
	`/api/github/analyze`		Analyze GitHub repo for licenses

Handler: `src/routes/github.js`

Request Body:

```
{
  "url": "https://github.com/owner/repo"
}
```

Response:

```
{
  "uuid": "550e8400-e29b-41d4-a716-446655440000",
  "overallVerdict": "NEEDS_REVIEW",
  "totalDependencies": 42,
  "dependencies": [
    {
      "name": "NPM::express:4.18.2",
      "version": "4.18.2",
      "license": "MIT",
      "licenseUrl": "https://spdx.org/licenses/MIT.html",
      "spdxExpression": "MIT"
    }
  ]
}
```

3.2 SPDX License Routes (`/api/spdx`)

Method	Endpoint	Controller	Purpose
GET	`/api/spdx/all`	`spdxRoutes`	Get all SPDX licenses
GET	`/api/spdx/search?q={query}`	`spdxRoutes`	Partial text search

Method	Endpoint	Controller	Purpose
GET	`/api/spdx/fuzzy?q={query}`	`spdxRoutes`	Fuzzy match (typo-tolerant)
GET	`/api/spdx/deprecated`	`spdxRoutes`	Get deprecated licenses
GET	`/api/spdx/stats`	`spdxRoutes`	License statistics

Handler: `src/routes/spdx.js`

3.3 Policy Compliance Routes (`/api/policy`)

Method	Endpoint	Controller	Purpose
POST	`/api/policy/check`	`policyRoutes`	Evaluate dependencies
GET	`/api/policy/check`	`policyRoutes`	Get latest cached report
GET	`/api/policy/check/:uuid`	`policyRoutes`	Get results by UUID
GET	`/api/policy/check/overall/:uuid`	`policyRoutes`	Get overall verdict only

Handler: `src/routes/policy.js`

3.4 Compliance Run Routes (`/api/compliance-runs`)

Method	Endpoint	Controller	Purpose
GET	`/api/compliance-runs`	`complianceRunsRoutes`	List all runs
GET	`/api/compliance-runs/:uuid`	`complianceRunsRoutes`	Get specific run

Method	Endpoint	Controller	Purpose
PUT	`/api/compliance-runs/:uuid`	`complianceRunsRoutes`	Update verdict

Handler: `src/routes/complianceRuns.js`

3.5 Shutdown Route (`/api/shutdown`)

Method	Endpoint	Middleware	Purpose
POST	`/api/shutdown`	`shutdownAuth`	Graceful shutdown

Handler: `src/routes/shutdown.js`

---

4. Service Layer Architecture

4.1 GitHubRepoService

File: `src/services/GitHubRepoService.js`

Responsibilities:
- Parse GitHub URLs
- Fetch repository metadata via GitHub API
- Download manifest files (package.json, pom.xml, etc.)
- Download lockfiles (package-lock.json, yarn.lock, etc.)
- Generate missing lockfiles (npm install --package-lock-only)
- Initialize Git repository for ORT compatibility
- Orchestrate ORT analysis

Key Methods:

```
analyzeRepository(repoUrl)          // Main entry point
parseGitHubUrl(url)                 // Extract owner/repo
findManifestFiles(tree)             // Find supported manifests
downloadLockfiles(...)              // Download lockfiles
generateNpmLockfile(workDir)        // Generate package-lock.json
```

```
initializeGitRepo(...)                // Create .git for ORT
analyzeInPlace(workDir)               // Run ORT analysis
```

Supported Manifest Files: 52+ types including:
- JavaScript: package.json, bower.json
- Java: pom.xml, build.gradle, build.gradle.kts
- Python: requirements.txt, setup.py, pyproject.toml, Pipfile
- Ruby: Gemfile, Gemfile.lock
- Go: go.mod, go.sum
- Rust: Cargo.toml, Cargo.lock
- PHP: composer.json
- .NET: *.csproj,* .fsproj, *.vbproj, packages.config*
- *Swift: Package.swift, Podfile, Cartfile*
- *Scala: build.sbt*
- *Haskell: stack.yaml, cabal.project*
- *Dart/Flutter: pubspec.yaml*
- *C/C++: conanfile.txt, conanfile.py, WORKSPACE, BUILD*
- *SPDX:* .spdx, *.spdx.json,* .spdx.yaml

4.2 LicenseAnalyzerService

File: `src/services/LicenseAnalyzerService.js`

Responsibilities:
- Execute ORT (OSS Review Toolkit) via Docker
- Create temporary working directories
- Parse ORT JSON output
- Extract dependency license information
- Handle ORT errors and warnings

Key Methods:

```
analyze(filePath)                     // Analyze manifest file
createWorkingDirectory()              // Create temp directory
executeOrt(workDir)                   // Run ORT Docker container
checkOrtIssues(analyzerResult)        // Check for errors
extractDependencies(analyzerResult)   // Parse ORT output
findSpdxUrl(spdxExpression)           // Resolve SPDX URLs
```

Docker Command Executed:

```
docker run --rm \
  -v /tmp/oss-analyzer/scan-{timestamp}:/workspace \
  ghcr.io/oss-review-toolkit/ort:latest \
  analyze \
  -i /workspace \
  -o /workspace/output \
  --output-formats JSON
```

4.3 ComplianceEvaluator

File: `src/services/ComplianceEvaluator.js`

Responsibilities:
- Evaluate dependencies against policy
- Parse SPDX license expressions
- Classify licenses as APPROVED, DENIED, or UNKNOWN
- Determine individual dependency verdicts (PASS, FAIL, REVIEW)
- Calculate overall compliance verdict

Evaluation Rules:

1. **FAIL (NON_COMPLIANT)** - Any denied license → immediate failure
2. **PASS (COMPLIANT)** - Exact SPDX expression match in approved list
3. **REVIEW (NEEDS_REVIEW)** - All sub-licenses approved but expression not listed
4. **REVIEW** - Unknown licenses present
5. **REVIEW** - Default case

Key Methods:

```
evaluate(dependencies)                // Main evaluation
evaluateDependency(dependency)        // Single dependency
extractLicenses(spdxExpression)       // Parse expression
classifyLicenses(licenses)            // APPROVED/DENIED/UNKNOWN
determineVerdict(...)                 // Apply policy rules
calculateOverallVerdict(verdicts)     // Aggregate result
```

Overall Verdict Logic:
- Any FAIL → `NON_COMPLIANT`

- Any REVIEW (no FAIL) → `NEEDS_REVIEW`
- All PASS → `COMPLIANT`

4.4 ComplianceRunService

File: `src/services/ComplianceRunService.js`

Responsibilities:
- Persist compliance reports to database
- Generate UUIDs for compliance runs
- Query compliance history
- Update verdicts

Key Methods:

```
saveComplianceRun(report, filePath)  // Save to DB
getAllRuns()                         // List all runs
getRunByUuid(uuid)                   // Get run with results
getResultsByUuid(uuid)               // Get verdicts only
getOverallVerdictByUuid(uuid)        // Get overall verdict
updateVerdict(uuid, newVerdict)      // Update verdict
```

4.5 SpdxLicenseRegistry

File: `src/services/SpdxLicenseRegistry.js`

Responsibilities:
- Load SPDX license database (500+ licenses)
- Provide fast O(1) license lookups
- Generate SPDX.org URLs

Data Source: `src/data/spdx/backup-license.json`

Key Methods:

```
initialize()                         // Load license JSON
getAllLicenses()                     // Return all licenses
findById(licenseId)                  // Case-insensitive lookup
exists(licenseId)                    // Check if license exists
getSpdxUrl(licenseId)                // Generate SPDX URL
```

---

5. Database Schema & Models

5.1 ComplianceRun Model

File: `src/models/ComplianceRun.js`

Table: `compliance_run`

Schema:

```
CREATE TABLE compliance_run (
  id UUID PRIMARY KEY,
  uuid UUID UNIQUE NOT NULL,
  file_path VARCHAR(500) NOT NULL,
  overall_verdict VARCHAR(50) NOT NULL,
  analyzed_at TIMESTAMP NOT NULL,
  validated_at TIMESTAMP,
  validation_verdict VARCHAR(50)
);

CREATE INDEX idx_uuid ON compliance_run(uuid);
CREATE INDEX idx_analyzed_at ON compliance_run(analyzed_at);
```

Fields:
- `id` - Internal primary key (UUID)
- `uuid` - External identifier for API queries
- `filePath` - GitHub URL or file path analyzed
- `overallVerdict` - COMPLIANT | NEEDS_REVIEW | NON_COMPLIANT
- `analyzedAt` - Timestamp of analysis
- `validatedAt` - Timestamp of manual validation
- `validationVerdict` - Updated verdict after review

5.2 PolicyComplianceResult Model

File: `src/models/PolicyComplianceResult.js`

Table: `policy_compliance_result`

Schema:

```
CREATE TABLE policy_compliance_result (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
  library VARCHAR(500) NOT NULL,
  spdx_expression VARCHAR(500) NOT NULL,
  compliance_result VARCHAR(20) NOT NULL,
  explanation TEXT NOT NULL,
  compliance_run_id UUID NOT NULL,
  FOREIGN KEY (compliance_run_id) REFERENCES compliance_run(id) ON DELETE CASC
);

CREATE INDEX idx_compliance_run_id ON policy_compliance_result(compliance_run_
CREATE INDEX idx_compliance_result ON policy_compliance_result(compliance_resu
```

Fields:
- `id` - Auto-increment primary key
- `library` - Full dependency identifier (e.g., "NPM::express:4.18.2")
- `spdxExpression` - License expression (e.g., "MIT", "Apache-2.0 OR MIT")
- `complianceResult` - PASS | FAIL | REVIEW
- `explanation` - Reason for verdict
- `complianceRunId` - Foreign key to ComplianceRun

5.3 Model Associations

File: `src/models/index.js`

```
ComplianceRun.hasMany(PolicyComplianceResult, {
  foreignKey: 'complianceRunId',
  as: 'results',
  onDelete: 'CASCADE'
});

PolicyComplianceResult.belongsTo(ComplianceRun, {
  foreignKey: 'complianceRunId',
  as: 'complianceRun'
});
```

---

6. Frontend Architecture (Alpine.js)

6.1 Main Analysis Page (`index.html`)

File: `public/index.html`

Alpine.js Component: `repoForm()`

State Variables:

```
repoUrl: ''                    // GitHub URL input
results: []                    // Dependency array
loading: false                 // Loading indicator
error: ''                      // Error message
uuid: ''                       // Compliance run UUID
overallVerdict: ''             // COMPLIANT/NEEDS_REVIEW/NON_COMPLIANT
totalDependencies: 0           // Dependency count
currentPage: 1                 // Pagination
perPage: 10                    // Items per page
```

User Actions:

1. **Enter GitHub URL** → Binds to `repoUrl`
2. **Click "Analyze Repository"** → Triggers `submitRepo()`
3. **View Results** → Displays paginated dependency table
4. **Download CSV** → Triggers `downloadCSV()`

API Calls:

```
POST /api/github/analyze      // Analyze repository
GET /api/spdx/stats           // Fetch license statistics (on init)
```

Flow:

```
User enters URL
    ↓
Click "Analyze Repository"
    ↓
submitRepo() → POST /api/github/analyze
    ↓
Server analyzes repo (30-120 seconds)
    ↓
```

```
Response: { uuid, overallVerdict, dependencies }
    ↓
Display results in table (paginated)
    ↓
Option to download CSV
```

6.2 SPDX License Browser (`licenses.html`)

File: `public/licenses.html`

Alpine.js Component: `licenseViewer()`

State Variables:

```
allLicenses: []              // All SPDX licenses
filteredLicenses: []         // Filtered results
searchQuery: ''              // Search input
currentPage: 1               // Pagination
pageSize: 10                 // Items per page
```

Features:
- Real-time search (filters by name or ID)
- Pagination (10 per page)
- Display FSF/OSI approval indicators with tooltips
- Links to SPDX.org for details
- Information alert explaining FSF and OSI approval

API Calls:

```
GET /api/spdx/all            // Load all licenses (on init)
```

6.3 PaaC Results Viewer (`paac-results.html`)

File: `public/paac-results.html`

Alpine.js Component: `paacResultsViewer()`

State Variables:

```
allResults: []               // Dependency verdicts
overallVerdict: ''           // Overall compliance
searchUuid: ''               // UUID search input
```

```
searchCompleted: false       // Search state
currentPage: 1               // Pagination
pageSize: 10                 // Items per page
```

Features:
- Search by UUID
- Display overall verdict banner
- Show dependency-level verdicts (PASS/FAIL/REVIEW)
- Paginated results

API Calls:

```
GET /api/policy/check                // Get latest cached results
GET /api/policy/check/{uuid}         // Get results by UUID
GET /api/policy/check/overall/{uuid} // Get overall verdict
```

6.4 Compliance Run Tracker (`compliance-run.html`)

File: `public/compliance-run.html`

Alpine.js Component: `complianceRunViewer()`

State Variables:

```
runs: []                     // Compliance run list
searchQuery: ''              // UUID filter
```

Features:
- List all compliance runs
- Filter by UUID (partial match)
- Update verdicts via dropdown
- Save updated verdicts

API Calls:

```
GET /api/compliance-runs             // List all runs (on init)
PUT /api/compliance-runs/{uuid}      // Update verdict
```

User Flow:

```
Page loads → Fetch all runs
    ↓
User filters by UUID
    ↓
Change verdict dropdown
    ↓
Click "Save" button
    ↓
PUT /api/compliance-runs/{uuid}
    ↓
Reload runs list
    ↓
Show success/error alert
```

6.5 Shutdown Control (`shutdown.html`)

File: `public/shutdown.html`

Alpine.js Component: `shutdownApp()`

State Variables:

```
apiKey: ''                    // Shutdown API key
message: ''                   // Status message
```

Features:
- API key authentication
- Graceful shutdown

API Calls:

```
POST /api/shutdown            // Initiate shutdown
```

---

7. Complete Request/Response Flow (Main Feature)

GitHub Repository Analysis - End-to-End Flow

```
┌─────────────────────────────────────────────────────────────────────────┐
│                          USER INTERACTION                               │
└─────────────────────────────────────────────────────────────────────────┘
                                    │
                     User enters GitHub URL in browser
                     (e.g., https://github.com/expressjs/express)
                                    │
                                    ▼
┌─────────────────────────────────────────────────────────────────────────┐
│                       FRONTEND (index.html)                             │
│  Alpine.js Component: repoForm()                                        │
└─────────────────────────────────────────────────────────────────────────┘
                                    │
                User clicks "Analyze Repository" button
                                    │
                    submitRepo() function executes
                                    │
                                    ▼
        POST /api/github/analyze
        Body: { url: "https://github.com/expressjs/express" }
                                    │
                                    ▼
┌─────────────────────────────────────────────────────────────────────────┐
│                      BACKEND ENTRY POINT                                │
│  File: src/server.js                                                    │
│  Middleware: helmet → cors → json → urlencoded → logger                 │
└─────────────────────────────────────────────────────────────────────────┘
                                    │
                     Route: /api/github/* → githubRoutes
                                    │
                                    ▼
┌─────────────────────────────────────────────────────────────────────────┐
│                       ROUTE HANDLER                                     │
│  File: src/routes/github.js                                             │
```

```
│  Endpoint: POST /api/github/analyze                                  │
└──────────────────────────────────────────────────────────────────────┘
                                  │
            1. Validate request body (express-validator)
            2. Extract URL from request
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────┐
│                    STEP 1: ANALYZE REPOSITORY                        │
│  Service: GitHubRepoService.analyzeRepository()                      │
│  File: src/services/GitHubRepoService.js                             │
└──────────────────────────────────────────────────────────────────────┘
                                  │
                     ┌────────────┴────────────┐
                     ▼                         ▼
        Parse GitHub URL          GitHub API Client
        owner: expressjs          (GitHubApiClient)
        repo: express                   │
                                        ▼
                            GET /repos/expressjs/express
                            (Fetch repo metadata)
                                        │
                                        ▼
                            GET /repos/expressjs/express/git/trees/master?recu
                            (Fetch repository tree - all files)
                                        │
                                        ▼
                     ┌──────────────────┴──────────────────┐
                     │  Find Manifest Files                 │
                     │  - package.json ✓                    │
                     │  - pom.xml ✗                         │
                     │  - requirements.txt ✗                │
                     └──────────────────┬──────────────────┘
                                        │
                                        ▼
                     Create Working Directory
                     /tmp/oss-analyzer/scan-1733445678901/
                                        │
                                        ▼
                     Download Manifest File
```

```
                    GET https://raw.githubusercontent.com/
                        expressjs/express/master/package.json
                                        |
                                        ▼
                    Check for Lockfiles
                    - package-lock.json ✓ (download)
                    - yarn.lock ✗
                                        |
                                        ▼
                    Initialize Git Repository
                    (ORT requires VCS info)
                    - git init
                    - git add .
                    - git commit -m "Initial commit"
                                        |
                                        ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                EXECUTE ORT (Docker Container)                            │
│  Service: LicenseAnalyzerService.executeOrt()                            │
│  File: src/services/LicenseAnalyzerService.js                            │
└──────────────────────────────────────────────────────────────────────────┘
                                  |
        Docker Command:
        docker run --rm \
          -v /tmp/oss-analyzer/scan-1733445678901:/workspace \
          ghcr.io/oss-review-toolkit/ort:latest \
          analyze \
          -i /workspace \
          -o /workspace/output \
          --output-formats JSON
                                  |
                    (ORT analyzes package.json + package-lock.json)
                    (Resolves all dependencies)
                    (Extracts license information)
                                  |
                                  ▼
                Output File Created:
                /tmp/oss-analyzer/scan-xxx/output/analyzer-result.json
                                  |
                                  ▼
```

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          PARSE ORT OUTPUT                                │
│  Method: extractDependencies(analyzerResult)                             │
└──────────────────────────────────────────────────────────────────────────┘
                                    │
        Extract from analyzer-result.json:
        {
          "analyzer": {
            "result": {
              "packages": [
                {
                  "id": "NPM::accepts:1.3.8",
                  "declared_licenses_processed": {
                    "spdx_expression": "MIT"
                  }
                },
                ...
              ]
            }
          }
        }
                                    │
                                    ▼
        Build Dependency Array:
        [
          {
            name: "NPM::accepts:1.3.8",
            version: "1.3.8",
            license: "MIT",
            licenseUrl: "https://spdx.org/licenses/MIT.html",
            spdxExpression: "MIT"
          },
          ...
        ]
                                    │
                                    ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                   STEP 2: EVALUATE COMPLIANCE                            │
│  Service: ComplianceEvaluator.evaluate()                                 │
│  File: src/services/ComplianceEvaluator.js                               │
```

```
└──────────────────────────────────────────────────────────────────────────────┘
                                  │
            Load Policy Configuration
            (from src/config/policy.js)
            - Approved: MIT, Apache-2.0, BSD-3-Clause, ...
            - Denied: SSPL-1.0, GPL-3.0, AGPL-3.0, ...
                                  │
                                  ▼
        For Each Dependency:
                                  │
            1. Extract licenses from SPDX expression
               "MIT" → ["MIT"]
               "Apache-2.0 OR MIT" → ["Apache-2.0", "MIT"]
                                  │
            2. Classify each license:
               - MIT → APPROVED ✓
               - SSPL-1.0 → DENIED ✗
               - Unknown-License → UNKNOWN ?
                                  │
            3. Determine verdict:
               - Any DENIED → FAIL
               - Exact expression match → PASS
               - All sub-licenses approved → REVIEW
               - Unknown licenses → REVIEW
                                  │
            4. Build verdict object:
               {
                 library: "NPM::express:4.18.2",
                 spdxExpression: "MIT",
                 complianceResult: "PASS",
                 explanation: "Exact match found in approved licenses."
               }
                                  │
                                  ▼
        Aggregate Overall Verdict:
        - Any FAIL → NON_COMPLIANT
        - Any REVIEW (no FAIL) → NEEDS_REVIEW
        - All PASS → COMPLIANT
                                  │
                                  ▼
```

```
        Compliance Report:
        {
          overallVerdict: "COMPLIANT",
          dependencyVerdicts: [
            { library: "...", complianceResult: "PASS", ... },
            ...
          ]
        }
                                  |
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│              STEP 3: SAVE TO DATABASE                                    │
│  Service: ComplianceRunService (implicit via route)                      │
│  Models: ComplianceRun, PolicyComplianceResult                           │
└──────────────────────────────────────────────────────────────────────────┘
                                  |
           1. Create ComplianceRun record:
              {
                uuid: "550e8400-e29b-41d4-a716-446655440000",
                filePath: "https://github.com/expressjs/express",
                overallVerdict: "COMPLIANT",
                analyzedAt: "2025-12-05T18:30:00.000Z"
              }
                                  |
           2. Create PolicyComplianceResult records (one per dependency):
              {
                library: "NPM::express:4.18.2",
                spdxExpression: "MIT",
                complianceResult: "PASS",
                explanation: "Exact match...",
                complianceRunId: "550e8400-..."
              }
                                  |
                   (Database: SQLite at ./data/paacdb.sqlite)
                                  |
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                  RETURN RESPONSE TO FRONTEND                             │
└──────────────────────────────────────────────────────────────────────────┘
                                  |
```

```
        HTTP 200 OK
        Content-Type: application/json

        {
          "uuid": "550e8400-e29b-41d4-a716-446655440000",
          "overallVerdict": "COMPLIANT",
          "totalDependencies": 42,
          "dependencies": [
            {
              "name": "NPM::accepts:1.3.8",
              "version": "1.3.8",
              "license": "MIT",
              "licenseUrl": "https://spdx.org/licenses/MIT.html",
              "spdxExpression": "MIT"
            },
            ...
          ]
        }
                                  |
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                     FRONTEND RENDERING                                   │
│  File: public/index.html                                                 │
│  Component: repoForm()                                                   │
└──────────────────────────────────────────────────────────────────────────┘
                                  |
            Update Alpine.js State:
            - uuid = "550e8400-..."
            - overallVerdict = "COMPLIANT"
            - totalDependencies = 42
            - results = [...]
                                  |
                                  ▼
            Render UI Components:

            1. Success Alert (Green Box):
               "✅ PaaC Evaluation Complete"
               UUID: 550e8400-...
               Overall Verdict: COMPLIANT (green badge)
               Dependencies: 42
```

```
            2. Dependency Table (Paginated):

               ┌──────────────┬──────────┬──────────┬──────────────┐
               │ Dependency   │ Version  │ License  │ License URL  │
               ├──────────────┼──────────┼──────────┼──────────────┤
               │ NPM::accep..│ 1.3.8    │ MIT      │ [SPDX Link]  │
               │ NPM::body-..│ 1.20.1   │ MIT      │ [SPDX Link]  │
               │ ...          │ ...      │ ...      │ ...          │
               └──────────────┴──────────┴──────────┴──────────────┘

               Page 1 of 5 [◀ Previous] [Next ▶]

            3. Download CSV Button:
               Clicking generates and downloads CSV file
                                  │
                                  ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                      ANALYSIS COMPLETE                                   │
│  User can now:                                                           │
│  - View detailed results                                                 │
│  - Download CSV report                                                   │
│  - Search results in PaaC Results page using UUID                        │
│  - Track run in Compliance Run Tracker                                   │
└──────────────────────────────────────────────────────────────────────────┘
```

---

8. Function Call Chains

8.1 GitHub Repository Analysis Call Chain

```
1. Frontend: submitRepo()
   └─> POST /api/github/analyze

2. Backend Route: src/routes/github.js
   └─> express-validator: validationResult()
   └─> githubRepoService.analyzeRepository(url)

3. GitHubRepoService.analyzeRepository()
```

```
   ├─> parseGitHubUrl(url)
   ├─> githubClient.getRepository(owner, repo)
   ├─> githubClient.getTree(owner, repo, branch)
   ├─> findManifestFiles(tree)
   ├─> createWorkingDirectory()
   ├─> githubClient.downloadFileToDirectory() [for each manifest]
   ├─> downloadLockfiles()
   ├─> generateNpmLockfile() [if needed]
   ├─> initializeGitRepo()
   └─> analyzeInPlace(workDir)
       ├─> licenseAnalyzer.executeOrt(workDir)
       │   └─> spawn('docker', [...])
       ├─> fs.readFileSync(analyzerResultPath)
       ├─> checkOrtIssues(analyzerResult)
       └─> licenseAnalyzer.extractDependencies(analyzerResult)
           └─> spdxRegistry.getSpdxUrl()

4. complianceEvaluator.evaluate(dependencies)
   ├─> evaluateDependency() [for each dependency]
   │   ├─> extractLicenses(spdxExpression)
   │   ├─> classifyLicenses(licenses)
   │   │   └─> policyConfig.isApproved() / isDenied()
   │   └─> determineVerdict()
   └─> calculateOverallVerdict(verdicts)

5. Database Persistence
   ├─> ComplianceRun.create({ uuid, filePath, overallVerdict })
   └─> PolicyComplianceResult.create() [for each verdict]

6. Response
   └─> res.json({ uuid, overallVerdict, totalDependencies, dependencies })
```

8.2 SPDX License Search Call Chain

```
Frontend: searchLicenses()
   └─> (Client-side filtering of allLicenses array)

Alternative: Server-side search
   └─> GET /api/spdx/search?q=MIT
```

```
        └─> spdxRoutes.get('/search')
            └─> spdxRegistry.getAllLicenses()
            └─> SpdxLicenseSearchUtil.searchByPartial(q, licenses)
                └─> licenses.filter(license =>
                      license.licenseId.toLowerCase().includes(query) ||
                      license.name.toLowerCase().includes(query)
                    )
```

8.3 Compliance Run Update Call Chain

```
Frontend: saveVerdict(uuid, verdict)
   └─> PUT /api/compliance-runs/{uuid}
       └─> complianceRunsRoutes.put('/:uuid')
           ├─> express-validator: validationResult()
           └─> complianceRunService.updateVerdict(uuid, verdict)
               ├─> ComplianceRun.findOne({ where: { uuid } })
               ├─> run.update({ overallVerdict, validatedAt, validationVerdict
               └─> run.toJSON()
```

---

9. API Endpoint Documentation

Complete API Reference

GitHub Analysis

POST /api/github/analyze

Analyze a GitHub repository for license compliance.

Request:

```
{
  "url": "https://github.com/owner/repo"
}
```

Response (200 OK):

```
{
  "uuid": "550e8400-e29b-41d4-a716-446655440000",
  "overallVerdict": "COMPLIANT | NEEDS_REVIEW | NON_COMPLIANT",
  "totalDependencies": 42,
  "dependencies": [
    {
      "name": "NPM::express:4.18.2",
      "version": "4.18.2",
      "license": "MIT",
      "licenseUrl": "https://spdx.org/licenses/MIT.html",
      "spdxExpression": "MIT"
    }
  ]
}
```

Errors:
- 400: Invalid GitHub URL
- 404: Repository not found
- 500: ORT analysis failed

SPDX License Queries

GET /api/spdx/all

Get all SPDX licenses.

Response (200 OK):

```
[
  {
    "licenseId": "MIT",
    "name": "MIT License",
    "isOsiApproved": true,
    "isFsfLibre": true,
    "isDeprecatedLicenseId": false
  }
]
```

GET /api/spdx/search?q={query}

Partial text search for licenses.

Query Params:
- `q` (required): Search query

Response (200 OK):

```
[
  {
    "licenseId": "MIT",
    "name": "MIT License",
    ...
  }
]
```

GET /api/spdx/fuzzy?q={query}

Fuzzy match license name (typo-tolerant).

Query Params:
- `q` (required): License name with potential typos

Response (200 OK):

```
{
  "licenseId": "MIT",
  "name": "MIT License",
  ...
}
```

Errors:
- 404: No fuzzy match found

GET /api/spdx/stats

Get SPDX registry statistics.

Response (200 OK):

```
{
  "totalLicenses": 534,
  "deprecatedLicenses": 12
}
```

Policy Compliance

POST /api/policy/check

Evaluate dependencies against compliance policy.

Request:

```
{
  "dependencies": [
    {
      "name": "express",
      "version": "4.18.2",
      "license": "MIT",
      "spdxExpression": "MIT"
    }
  ],
  "filePath": "path/to/package.json"
}
```

Response (200 OK):

```
{
  "uuid": "550e8400-e29b-41d4-a716-446655440000",
  "overallVerdict": "COMPLIANT",
  "dependencyVerdicts": [
    {
      "library": "express",
      "spdxExpression": "MIT",
      "complianceResult": "PASS",
      "explanation": "Exact match found in approved licenses."
    }
```

```
  ]
}
```

GET /api/policy/check

Get latest compliance report from cache.

Response (200 OK):

```
{
  "uuid": "550e8400-...",
  "overallVerdict": "COMPLIANT",
  "dependencyVerdicts": [...]
}
```

Errors:
- 404: No compliance report available

GET /api/policy/check/:uuid

Get detailed results for a compliance run.

URL Params:
- `uuid` (required): Compliance run UUID

Response (200 OK):

```
[
  {
    "id": 1,
    "library": "NPM::express:4.18.2",
    "spdxExpression": "MIT",
    "complianceResult": "PASS",
    "explanation": "Exact match found..."
  }
]
```

GET /api/policy/check/overall/:uuid

Get overall verdict for a compliance run.

URL Params:
- `uuid` (required): Compliance run UUID

Response (200 OK):

```
COMPLIANT
```

(Plain text response)

Compliance Run Management

GET /api/compliance-runs

List all compliance runs.

Response (200 OK):

```
[
  {
    "uuid": "550e8400-...",
    "filePath": "https://github.com/owner/repo",
    "overallVerdict": "COMPLIANT",
    "analyzedAt": "2025-12-05T18:30:00.000Z",
    "validatedAt": null,
    "validationVerdict": null
  }
]
```

GET /api/compliance-runs/:uuid

Get specific compliance run with results.

URL Params:
- `uuid` (required): Compliance run UUID

Response (200 OK):

```
{
  "uuid": "550e8400-...",
  "filePath": "...",
  "overallVerdict": "COMPLIANT",
  "analyzedAt": "...",
  "results": [
    {
      "id": 1,
      "library": "...",
      "spdxExpression": "MIT",
      "complianceResult": "PASS",
      "explanation": "..."
    }
  ]
}
```

PUT /api/compliance-runs/:uuid

Update overall verdict for a compliance run.

URL Params:
- `uuid` (required): Compliance run UUID

Request:

```
{
  "overallVerdict": "COMPLIANT | NEEDS_REVIEW | NON_COMPLIANT"
}
```

Response (200 OK):

```
{
  "uuid": "550e8400-...",
  "overallVerdict": "NEEDS_REVIEW",
  "validatedAt": "2025-12-05T19:00:00.000Z",
  "validationVerdict": "NEEDS_REVIEW"
}
```

System Control

POST /api/shutdown

Gracefully shutdown the application.

Headers:
- `X-AIG-SHUTDOWN-KEY` or `X-API-KEY` (required): Shutdown API key

Response (200 OK):

```
{
  "message": "Shutdown initiated"
}
```

Errors:
- 401: Unauthorized (invalid API key)

GET /health

Health check endpoint.

Response (200 OK):

```
{
  "status": "ok",
  "timestamp": "2025-12-05T18:30:00.000Z",
  "uptime": 3600.5
}
```

---

10. User Journey Mapping

Journey 1: Analyze GitHub Repository

```
┌──────────────────────────────────────────────────────────────────────┐
│ Actor: Developer                                                     │
│ Goal: Analyze a GitHub repository for license compliance             │
```

```
└──────────────────────────────────────────────────────────────────────┘

1. Navigate to http://localhost:8080
   └─> Page: index.html
   └─> Component loads: repoForm()
   └─> Initialization: Fetch SPDX stats (GET /api/spdx/stats)

2. View homepage
   └─> See: "AIG OSS License Analyzer" header
   └─> See: GitHub URL input field
   └─> See: Navigation buttons (PaaC Results, Compliance Tracker, etc.)
   └─> See: Footer showing "SPDX Registry contains 534 licenses"

3. Enter GitHub URL
   └─> Input: "https://github.com/expressjs/express"
   └─> Alpine.js: x-model="repoUrl" binds value

4. Click "Analyze Repository" button
   └─> Alpine.js: @click="submitRepo()"
   └─> Function executes:
       - Set loading = true
       - Clear previous results
       - POST /api/github/analyze { url: repoUrl }

5. Wait for analysis (30-120 seconds)
   └─> See: "Analyzing... Please wait." message
   └─> Backend:
       - Parse GitHub URL
       - Fetch repo metadata
       - Download manifests
       - Run ORT in Docker
       - Evaluate compliance
       - Save to database

6. View results
   └─> Success alert appears:
       ┌──────────────────────────────────────────┐
       │ ✅ PaaC Evaluation Complete               │
       │ Compliance UUID: 550e8400-...             │
       │ Overall Verdict: COMPLIANT (green badge)  │
```

```
        │ Dependencies: 42                          │
        └──────────────────────────────────────────┘

    └─> Dependency table renders:
        - Shows 10 dependencies per page
        - Columns: Dependency, Version, License, License URL
        - Pagination controls at bottom
        - "Download CSV" button at top

7. Interact with results
   Option A: Download CSV
   └─> Click "Download CSV" button
   └─> Function: downloadCSV()
   └─> Browser downloads "oss-license-analysis.csv"

   Option B: View detailed compliance
   └─> Copy UUID from success alert
   └─> Click "PaaC Results" navigation button
   └─> Navigate to paac-results.html
   └─> Paste UUID in search box
   └─> View compliance verdicts (PASS/FAIL/REVIEW)

   Option C: Track in Compliance Tracker
   └─> Click "Compliance Tracker" navigation button
   └─> Navigate to compliance-run.html
   └─> Find run by UUID
   └─> Update verdict if needed
```

Journey 2: Browse SPDX Licenses

```
┌──────────────────────────────────────────────────────────────────────┐
│ Actor: Compliance Manager                                            │
│ Goal: Explore SPDX licenses and check approval status                │
└──────────────────────────────────────────────────────────────────────┘

1. Navigate to http://localhost:8080
   └─> Click "SPDX Licenses" button in navigation
```

```
2. Page loads: licenses.html
   └─> Component: licenseViewer()
   └─> Initialization: GET /api/spdx/all
   └─> Load all 534 licenses into memory

3. View informational alert
   └─> See: Blue info box explaining FSF and OSI approval
   └─> Learn: What FSF Approved and OSI Approved mean

4. View license table
   └─> See: Paginated table (10 per page)
   └─> Columns:
       - License Name (clickable → SPDX.org)
       - License ID
       - FSF Approved ℹ️ (✓ or ✗, hover for tooltip)
       - OSI Approved ℹ️ (✓ or ✗, hover for tooltip)

5. Search for specific license
   └─> Type "MIT" in search box
   └─> Alpine.js: @input="searchLicenses()"
   └─> Client-side filtering executes
   └─> Table updates in real-time
   └─> Shows: "Showing 3 licenses (filtered from 534)"

6. View license details
   └─> Click on license name (e.g., "MIT License")
   └─> Opens: https://spdx.org/licenses/MIT.html
   └─> View full license text and metadata

7. Navigate pages
   └─> Click "Next" button
   └─> currentPage increments
   └─> Table shows next 10 results
```

Journey 3: Review Compliance Results

```
┌──────────────────────────────────────────────────────────────────┐
│ Actor: Security Auditor                                          │
│ Goal: Review compliance verdicts for a specific analysis run     │
└──────────────────────────────────────────────────────────────────┘

1. Receive UUID from developer
   └─> UUID: "550e8400-e29b-41d4-a716-446655440000"

2. Navigate to PaaC Results page
   └─> URL: http://localhost:8080/paac-results.html
   └─> Component: paacResultsViewer()
   └─> Initialization: GET /api/policy/check (latest cached)

3. Search by UUID
   └─> Paste UUID in search box
   └─> Press Enter or click "Search"
   └─> Function: loadResultsByUUID(uuid)
   └─> API Calls:
       - GET /api/policy/check/{uuid}
       - GET /api/policy/check/overall/{uuid}

4. View overall verdict
   └─> Banner displays:
       ┌──────────────────────────────┐
       │ Overall Compliance Verdict   │
       │                              │
       │      NEEDS REVIEW            │
       │      (yellow/warning color)  │
       └──────────────────────────────┘

5. Review dependency-level verdicts
   └─> Table shows:
       ┌────────────┬─────────┬──────────────┬─────────┬────────────┐
       │ Library    │ Version │ SPDX Expr.   │ Verdict │ Explanation│
       ├────────────┼─────────┼──────────────┼─────────┼────────────┤
       │ express    │ -       │ MIT          │ PASS    │ Exact mat..│
       │ body-p...  │ -       │ MIT          │ PASS    │ Exact mat..│
       │ some-lib   │ -       │ GPL-3.0      │ FAIL    │ Contains d.│
```

```
        └──────────┴─────────┴──────────────┴─────────┴──────────┘

   └─> Color coding:
       - PASS → Green
       - FAIL → Red
       - REVIEW → Yellow/Orange

6. Identify problematic dependencies
   └─> Scan for FAIL verdicts
   └─> Read explanation: "Contains denied license(s): GPL-3.0"
   └─> Note libraries for remediation

7. Navigate to Compliance Tracker
   └─> Click "Compliance Tracker" button
   └─> Filter by UUID
   └─> Update overall verdict if after manual review
```

Journey 4: Update Compliance Verdict

```
┌──────────────────────────────────────────────────────────────────┐
│ Actor: Compliance Manager                                        │
│ Goal: Update compliance verdict after manual review              │
└──────────────────────────────────────────────────────────────────┘

1. Navigate to Compliance Run Tracker
   └─> URL: http://localhost:8080/compliance-run.html
   └─> Component: complianceRunViewer()
   └─> Initialization: GET /api/compliance-runs

2. View all compliance runs
   └─> Table shows:
       - UUID
       - Created At
       - Path (GitHub URL)
       - Overall Verdict (dropdown)
       - Action (Save button)
```

```
3. Filter by UUID
   └─> Type partial UUID in filter box
   └─> Alpine.js: filteredRuns() filters list
   └─> Table updates in real-time

4. Update verdict
   └─> Change dropdown from "NEEDS_REVIEW" to "COMPLIANT"
   └─> Alpine.js: x-model="run.updatedVerdict"
   └─> Click "Save" button

5. Save to database
   └─> Function: saveVerdict(uuid, verdict)
   └─> API Call: PUT /api/compliance-runs/{uuid}
   └─> Request body: { overallVerdict: "COMPLIANT" }

6. Confirm update
   └─> Alert: "Verdict updated successfully!"
   └─> Table reloads: GET /api/compliance-runs
   └─> See updated verdict in table
```

Journey 5: Graceful Shutdown

```
┌──────────────────────────────────────────────────────────────────────┐
│ Actor: System Administrator                                          │
│ Goal: Gracefully shutdown the application                            │
└──────────────────────────────────────────────────────────────────────┘

1. Navigate to Shutdown page
   └─> URL: http://localhost:8080/shutdown.html
   └─> Component: shutdownApp()

2. Enter API key
   └─> Input field: x-model="apiKey"
   └─> Enter: "dummy-key" (from .env)

3. Click "Shutdown Application" button
   └─> Function: shutdown()
```

```
   └─> API Call: POST /api/shutdown
   └─> Headers: { 'X-AIG-SHUTDOWN-KEY': apiKey }

4. Backend processes shutdown
   └─> Middleware: shutdownAuth validates API key
   └─> Route handler: shutdown.js
   └─> Response sent: { message: "Shutdown initiated" }
   └─> setTimeout(1000): Close database connections
   └─> process.exit(0)

5. View shutdown message
   └─> Alert: "Shutdown initiated successfully. The app will exit shortly."
   └─> Application terminates after 1 second
```

---

11. Configuration Files

Environment Variables (.env)

```
# Server
PORT=8080
NODE_ENV=development

# Security
SHUTDOWN_API_KEY=dummy-key

# GitHub API
GITHUB_TOKEN=ghp_xxxxx  # Personal access token

# Analysis
ANALYZER_WORK_DIR=/tmp/oss-analyzer

# Policy Configuration
APPROVED_LICENSES=MIT,Apache-2.0,BSD-2-Clause,BSD-3-Clause,ISC,EPL-2.0,MPL-2.0
DENIED_LICENSES=SSPL-1.0,BSL-1.1,RPL-1.5,GPL-3.0,AGPL-3.0

# Database
DATABASE_PATH=./data/paacdb.sqlite
```

```
# Docker
DOCKER_IMAGE=ghcr.io/oss-review-toolkit/ort:latest
DOCKER_WORKSPACE=/workspace

# Logging
LOG_LEVEL=info
LOG_FILE=./logs/app.log
```

---

12. Key Technologies & Dependencies

Backend Stack

- **Runtime:** Node.js 18+
- **Framework:** Express.js 4.18.2
- **Database:** SQLite 5.1.6 + Sequelize ORM 6.35.0
- **Logging:** Winston 3.11.0
- **HTTP Client:** Axios 1.6.0
- **Validation:** express-validator 7.0.1
- **Security:** Helmet 7.1.0, CORS 2.8.5
- **String Matching:** fastest-levenshtein 1.0.16 (fuzzy search)
- **UUID Generation:** uuid 9.0.1

Frontend Stack

- **Framework:** Alpine.js 3.x (CDN)
- **Styling:** Bootstrap 5.3.3 (CDN)
- **Custom CSS:** aig-bootstrap-override.css

External Tools

- **License Analysis:** OSS Review Toolkit (ORT) via Docker
- Image: `ghcr.io/oss-review-toolkit/ort:latest`
- Function: Analyze dependency manifests, resolve licenses
- **Package Management:** npm (for generating lockfiles)

---

13. Data Flow Summary

```
GitHub Repository URL
          ↓
GitHub API (fetch metadata, tree, files)
          ↓
Working Directory (/tmp/oss-analyzer/scan-{timestamp})
          ↓
ORT Docker Container (analyze dependencies)
          ↓
ORT Output (analyzer-result.json)
          ↓
Dependency Extraction (parse JSON)
          ↓
Policy Evaluation (compare against approved/denied lists)
          ↓
Compliance Report (overall verdict + individual verdicts)
          ↓
Database Persistence (SQLite)
          ↓
HTTP Response (JSON)
          ↓
Frontend Rendering (Alpine.js)
          ↓
User Interface (tables, badges, pagination)
```

---

14. Security Considerations

1. **API Key Protection**
   - Shutdown endpoint requires `X-AIG-SHUTDOWN-KEY` header
   - Middleware: `shutdownAuth.js` validates key
2. **Input Validation**
   - `express-validator` validates all request bodies
   - GitHub URL parsing prevents malicious inputs

3. **Security Headers**
   - Helmet middleware adds security headers
   - CSP disabled to allow inline Alpine.js scripts
4. **CORS**
   - Enabled for cross-origin requests
   - Configure as needed for production
5. **GitHub Token**
   - Store in `.env` file (not in source control)
   - Use for authenticated API requests (higher rate limits)
6. **Docker Isolation**
   - ORT runs in ephemeral containers (`--rm` flag)
   - Volume mounts limited to working directory
7. **Error Handling**
   - Global error handler catches exceptions
   - Logs errors with Winston
   - Returns sanitized error messages to clients

---

15. Performance Characteristics

Analysis Time

- **Small repos (<50 deps):** 30-60 seconds
- **Medium repos (50-200 deps):** 60-120 seconds
- **Large repos (>200 deps):** 2-5 minutes

Bottlenecks

1. **ORT Docker Execution** - Most time-consuming step
2. **GitHub API Rate Limits** - 60 req/hr (unauthenticated), 5000 req/hr (authenticated)
3. **Lockfile Generation** - npm install can be slow

Optimization Opportunities

- Cache ORT Docker image locally
- Use GitHub token for higher rate limits
- Parallelize manifest downloads

- Database indexing on frequently queried fields

Conclusion

This OSS License Analyzer provides a complete end-to-end solution for analyzing GitHub repositories for open-source license compliance. The architecture follows clean separation of concerns with:

- **Routes** handling HTTP requests
- **Services** containing business logic
- **Models** representing database entities
- **Utilities** providing reusable functions
- **Middleware** for cross-cutting concerns
- **Frontend** using reactive Alpine.js components

The system integrates with industry-standard tools (ORT, SPDX) and provides a user-friendly web interface for compliance management. All major user journeys are mapped from UI interaction through the full backend stack to database persistence and back to the UI.